UNIVERSITY OF MINDANAO

COLLEGE OF COMPUTING EDUCATION

IT6/L – FUNDAMENTALS OF DATABASE



[UNIVERSITY OF SAINT FREYA, STUDENT ENROLLMENT SYSTEM]

FINAL PROJECT TECHNICAL DOCUMENTATION

Submitted by:

Ryu Booc Sun

Jul Khair Umali

Submitted to:

Jhun Bart Macirin

IT6/L 3438

[Submission Date]

Chapter 1. Project Introduction

1.1 Project Title

University of Saint Freya, Student Enrollment System

1.2 Background of the Study / Existing Process

The process of enrollment is an essential part of every educational institution. It ensures that students are registered properly and assigned to their respective academic courses and programs. Universities handle a wide array of student information, this includes academic backgrounds, personal records and achievements, details of the program, offering for subjects and records of the enrollment themselves. The management of these records are important to maintain an organized academic operation while providing the students with a smooth enrollment experience.

Universities around the world continue to accommodate growing academic programs and the student population itself which in turn makes managing enrollment information become increasingly demanding. Traditional or poorly integrated recordkeeping processes may lead to difficulties in organizing student information, monitoring records of enrollment, maintaining accurate academic data, and retrieving back information when needed. These challenges highlight the importance of having a well-structured and centralized database that supports the institution's enrollment operations.

The University of Saint Freya is a hypothetical educational institution established as the setting of this project. Its proposed Student Enrollment Systems aims to provide an organized approach to managing student registration, academic programs, course offerings and enrollment records. The system shall utilize a relational database to keep and maintain consistency, accuracy, and proper relationships among the institution's administrative and academic information.

Through the development of this system, the project seeks to demonstrate how proper database design and implementation can support university enrollment operations. It will

focus on identifying the institution's data requirements, establishing appropriate database relationships, applying normalization principles, and implementing the database to provide a reliable foundation for managing enrollment information.

1.2.1 Existing Process

Currently, the University of Saint Freya follows a traditional and manual enrollment process in which students are required to coordinate with the university's administrative offices to complete their enrollment. The process consists of several stages, beginning with student registration and ending with the confirmation of the enrollment.

1. **Student Registration** → Students obtain and complete registration forms containing their personal and academic information.
2. **Document Verification** → The registrar's office reviews the submitted documents and verifies the student's information.
3. **Academic Advising** → Students coordinate with their respective academic departments regarding their eligible subjects and academic schedules.
4. **Subject Enrollment** → Enrollment personnel record the students' selected subjects and relevant academic information.
5. **Assessment and Payment** → Students receive their enrollment assessment and settle the applicable fees.
6. **Enrollment Confirmation** → Administrative personnel verify the completion of enrollment requirements and confirm the student's enrollment status.

Although this process allows the university to manage student enrollment, relying on physical documents and separate digital records may result in redundant and repetitive data entry, delays in retrieving and receiving information and difficulties maintaining student and academic records.

The proposed Student Enrollment System aims to address these challenges by providing an online platform for managing student registration and academic records. By transitioning from physical documentation to an organized digital process, the system seeks to improve

data entry accuracy, minimize redundant information, and simplify the maintenance and retrieval of student and academic records. Furthermore, centralizing these records within a relational database will help maintain data consistency and support more efficient enrollment operations.

1.3 Statement of the Problem

The existing enrollment process of the University of Saint Freya presents several challenges in managing student information and academic records. Reliance on physical documentation and separate recordkeeping procedures may result in redundant data entry, difficulties in retrieving information, and inconsistencies in maintaining student records. These challenges highlight the need for a more organized and centralized enrollment system.

Specifically, this project seeks to address the following problems:

→ How can student information and registration records be organized and maintained efficiently?
→ How can academic programs, subjects, instructors, and sections be properly managed within a centralized database?
→ How can redundant and inconsistent data be minimized through proper database design and normalization?
→ How can enrollment transactions and payment records be accurately recorded and monitored?
→ How can authorized university personnel efficiently retrieve and manage student and academic records?

1.4 General and Specific Objectives

To design and develop a Student Enrollment System for the University of Saint Freya that provides an organized and centralized platform for managing student records, academic programs, subject offerings, section assignments, enrollment transactions, and payment records through a relational database.

The project specifically aims to:

1. To develop a structured database for managing university departments, academic programs, subjects, instructors, and student information.
2. To organize subject offerings into sections with corresponding instructors, schedules, classrooms, academic terms, and enrollment capacities.
3. To establish a systematic process for recording student enrollments in available class sections.
4. To maintain student payment records, including payment amounts, dates, and corresponding payment methods.
5. To establish appropriate relationships between database entities through primary keys, foreign keys, and constraints.
6. To apply database normalization principles to minimize data redundancy and maintain data consistency.
7. To implement and test the database using Laravel and a relational database management system.

1.5 Scope and Limitations

1.5.1 Scope

The proposed Student Enrollment System for the University of Saint Freya focuses on organizing and managing student information, academic records, enrollment transactions, and payment information through a centralized relational database. The system covers the management of departments, academic programs, subjects, instructors, students, class sections, enrollments, and payment records.

Furthermore, the database is designed to establish relationships between these entities to ensure organized data storage and efficient record management. It also includes the recording of class schedules, assigned instructors, enrollment capacities, student enrollment statuses, and payment transactions. The project will involve database design, implementation, and testing using Laravel and a relational database management system.

1.5.2 Limitations

The proposed Student Enrollment System is primarily focused on managing student information, academic records, enrollment transactions, and payment information. It does not cover other university operations such as grade management, attendance monitoring, examination records, library services, or other administrative functions outside the scope of enrollment management.

Furthermore, the system's payment functionality is limited to recording payment information and does not initially support direct integration with external banking institutions or online payment gateways. Advanced academic scheduling features, including automatic conflict detection and timetable optimization, are also not included in the initial implementation.

The project is developed specifically for the hypothetical University of Saint Freya and will focus on the requirements and database structure established for this academic project.

1.6 Intended Users and Benefits

1.6.1 Intended Users

The proposed Student Enrollment System is intended for individuals responsible for managing and participating in the enrollment operations of the University of Saint Freya. These include students, registrar personnel, academic department personnel, finance personnel, and system administrators. Each user is expected to interact with the system according to their respective responsibilities and access permissions.

1. Students
2. Registrar Personnel
3. Academic Department Personnel
4. Finance Personnel
5. System Administrators

1.6.2 Benefits of the System

The implementation of the proposed Student Enrollment System is expected to provide several benefits to the University of Saint Freya and its intended users. Through a centralized

database, the system aims to improve the organization, maintenance, and retrieval of student and academic records while reducing redundant data entry and inconsistencies.

1. Benefits to Students
2. Benefits to Registrar Personnel
3. Benefits to Academic Department Personnel
4. Benefits to Finance Personnel
5. Benefits to the University Administration

Chapter 2. Requirements Analysis

2.1 Description of the Existing Process

The existing enrollment process of the University of Saint Freya follows a traditional, manually coordinated procedure consisting of six steps, beginning with student registration and concluding with enrollment confirmation. Throughout this process, students are required to coordinate with the university's administrative personnel and provide the necessary documents and personal information to complete their enrollment requirements.

→ 1. Student Registration
→ 2. Document Verification
→ 3. Academic Advising
→ 4. Subject Enrollment
→ 5. Assessment and Payment
→ 6. Enrollment Confirmation

2.2 Data Gathering Method and Findings

2.2.1 Data Gathering Methods

The initial requirements of the proposed Student Enrollment System were established through multiple data-gathering methods. These methods include research of related systems, document analysis, group discussions, and database design analysis. Each method contributed

to identifying the necessary information, processes, and requirements for the proposed system.

The data-gathering process for the proposed system consists of the following procedures:

First Procedure: Identification of Information Requirements to help determine what information is necessary for university enrollment operations.
Second Procedure: Research of Related Systems which examine existing university enrollment systems to understand their common functions and processes.
Third Procedure: Analysis of Related Documents in order to review available examples of resignation forms, academic records, and payment documentation.
Fourth Procedure: Group Discussion and Evaluation. This is to discuss the gathered information and determine which requirements are relevant to the University of Saint Freya.
Fifth Procedure: Database Requirements Identification so that the identified requirements will be translated into entities, attributes, relationships, and constraints.

2.2.2 Findings

- Student information must be systematically maintained.
- Academic programs and subjects must be properly organized.
- Sections must be associated with instructors and subjects.
- Enrollment records must connect students with their selected sections.
- Payment transactions must be properly recorded.

Based on the findings gathered through the identified methods, the proposed Student Enrollment System requires a structured and centralized database to support the management of student information, academic records, enrollment transactions, and payment information. These findings served as the basis for identifying the necessary database entities, attributes, and relationships that will be incorporated into the system's design and implementation.

2.3 Functional Requirements

ID	Functional Requirement	Description
FR-01	Student Registration	The system shall allow student information to be recorded and stored.
FR-02	Student Information Management	The system shall allow student records to be viewed and updated.
FR-03	Department Management	The system shall allow department information to be maintained.
FR-04	Academic Program Management	The system shall allow academic programs to be recorded and associated with departments.
FR-05	Subject Management	The system shall allow subjects and their corresponding units to be managed.
FR-06	Instructor Management	The system shall allow instructor information to be recorded and maintained.
FR-07	Section Management	The system shall allow sections, schedules, assigned instructors, and maximum capacities to be managed.
FR-08	Student Enrollment	The system shall allow students to enroll in available sections.
FR-09	Enrollment Record Management	The system shall allow enrollment records and their statuses to be viewed and updated.
FR-10	Payment Management	The system shall allow student payments to be recorded and maintained.
FR-11	Payment Method Management	The system shall allow available payment methods to be managed.

2.4 Business Rules

ID	Business Rule
BR-01	Each department must have a unique department code.
BR-02	Each academic program must belong to an associated department.

BR-03	Each subject must be associated with a department.
BR-04	Each instructor must be associated with a department.
BR-05	Each student must have a unique student number.
BR-06	Each student may be associated with an academic program.
BR-07	Each section must correspond to an existing subject and instructor.
BR-08	Each section must have a defined maximum enrollment capacity.
BR-09	An enrollment record must reference an existing student and section.
BR-10	A student's enrollment status must follow the system's predefined enrollment statuses.
BR-11	Each payment must be associated with an existing student.
BR-12	Each payment must use an existing payment method.
BR-13	Each payment must have a recorded amount, school year, and semester.
BR-14	Payment methods must have an availability status indicating whether they can be used.

Chapter 3. Database Design

3.1 Identified Entities and Attributes

The system has nine entities. Three describe the school's structure (departments, programs, subjects), two describe people (students, instructors), and four record activity (sections, enrollments, payments, payment methods). Primary keys are underlined in the ERD and marked PK below; foreign keys are marked FK.

Table 3.1. Identified entities and attributes

Entity	**Description**	**Attributes**
departments	Academic departments of the institution	**department_id (PK)**, department_code, department_name

programs	Degree programs offered by a department	**program_id (PK)**, program_code, program_name, department_id (FK)
subjects	Courses that can be offered, owned by a department	**subject_id (PK)**, subject_code, subject_name, units, department_id (FK)
instructors	Faculty members who handle sections	**instructor_id (PK)**, first_name, middle_name, last_name, email, department_id (FK)
students	Students registered in a program	**student_id (PK)**, student_no, first_name, middle_name, last_name, gender, email, contact_no, program_id (FK), year_level, status, date_registered
sections	A class offering of one subject for a school year and semester	**section_id (PK)**, section_code, subject_id (FK), instructor_id (FK), school_year, semester, schedule_day, start_time, end_time, room, max_capacity
enrollments	A student's enrollment in one section	**enrollment_id (PK)**, student_id (FK), section_id (FK), enrollment_datetime, status
payments	A payment made by a student for a school year and semester	**payment_id (PK)**, student_id (FK), school_year, semester, amount, payment_method_id (FK), payment_datetime
payment_methods	Accepted ways of paying (e.g., cash, GCash, bank transfer)	**payment_method_id (PK)**, method_name, method_available

3.2 Entity-Relationship Diagram (ERD)

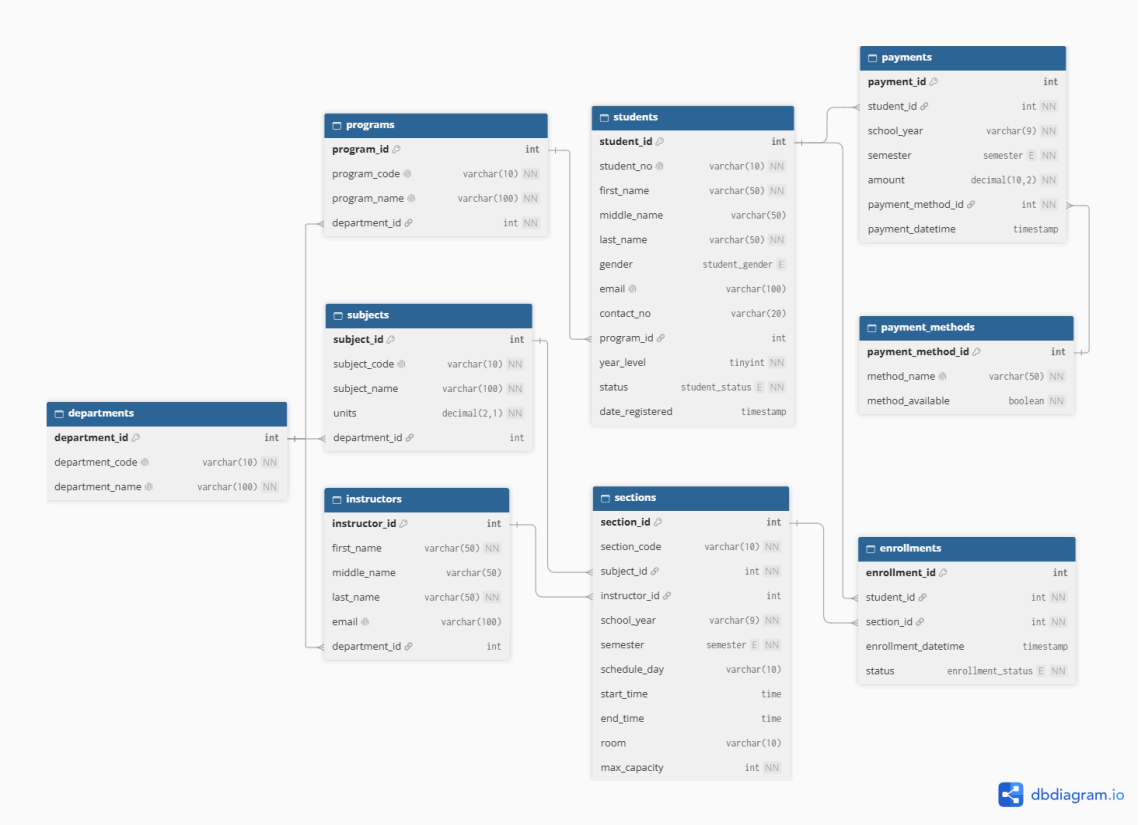


Figure 3.1. Entity-Relationship Diagram of the system

3.3 Relational Schema

Each relation is written as TABLE(attributes). Primary keys are in **bold**, and foreign keys are in *italics* followed by the table they reference.

1. departments(**department_id**, department_code, department_name)
2. programs(**program_id**, program_code, program_name, *department_id* → departments)
3. subjects(**subject_id**, subject_code, subject_name, units, *department_id* → departments)
4. instructors(**instructor_id**, first_name, middle_name, last_name, email, *department_id* → departments)
5. students(**student_id**, student_no, first_name, middle_name, last_name, gender, email, contact_no, *program_id* → programs, year_level, status, date_registered)
6. sections(**section_id**, section_code, *subject_id* → subjects, *instructor_id* → instructors, school_year, semester, schedule_day, start_time, end_time, room, max_capacity)

7. enrollments(**enrollment_id**, *student_id* → students, *section_id* → sections, enrollment_datetime, status)
8. payment_methods(**payment_method_id**, method_name, method_available)
9. payments(**payment_id**, *student_id* → students, school_year, semester, amount, *payment_method_id* → payment_methods, payment_datetime)

3.4 Database Normalization (1NF, 2NF, 3NF)

The database was normalized from a single unnormalized student record into nine tables in Third Normal Form (3NF). The starting point is the information currently written on one student's enrollment and payment form.

3.4.1 Unnormalized Form (UNF)

STUDENT_RECORD(student_no, student_name, gender, email, contact_no, program_code, program_name, department_code, department_name, year_level, status, {subject_code, subject_name, units, section_code, school_year, semester, schedule, room, instructor_name, instructor_email, enrollment_status}, {payment_date, amount, payment_method})

Problems in UNF:

- The subjects a student takes and the payments a student makes are repeating groups (shown in braces).
- student_name, instructor_name and schedule (e.g., "MWF 8:00–9:00") are not atomic.
- Program, department, subject and instructor details are repeated in every student's record.

3.4.2 First Normal Form (1NF)

A table is in 1NF when every field holds one atomic value and there are no repeating groups. To reach 1NF:

- Names were split into first_name, middle_name and last_name.
- schedule was split into schedule_day, start_time and end_time.
- Each repeating group was moved to its own row, giving two relations:

1. STUDENT_ENROLLMENT(student_no, section_code, school_year, semester, first_name, middle_name, last_name, gender, email, contact_no, program_code, program_name, department_code, department_name, year_level, status, subject_code, subject_name, units, schedule_day, start_time, end_time, room, max_capacity, instructor_first_name, instructor_last_name, instructor_email, enrollment_datetime, enrollment_status)
2. STUDENT_PAYMENT(student_no, payment_datetime, first_name, last_name, school_year, semester, amount, method_name, method_available)

3.4.3 Second Normal Form (2NF)

A table is in 2NF when it is in 1NF and no attribute depends on only part of a composite key. In STUDENT_ENROLLMENT, student details depend only on student_no, and section details depend only on (section_code, school_year, semester). In STUDENT_PAYMENT, the student's name depends only on student_no. Removing these partial dependencies gives:

1. STUDENT(student_no, first_name, middle_name, last_name, gender, email, contact_no, program_code, program_name, department_code, department_name, year_level, status, date_registered)
2. SECTION(section_code, school_year, semester, subject_code, subject_name, units, subject_department, instructor_first_name, instructor_last_name, instructor_email, schedule_day, start_time, end_time, room, max_capacity)
3. ENROLLMENT(student_no, section_code, school_year, semester, enrollment_datetime, status)
4. PAYMENT(student_no, payment_datetime, school_year, semester, amount, method_name, method_available)

3.4.4 Third Normal Form (3NF)

A table is in 3NF when it is in 2NF and no non-key attribute depends on another non-key attribute (no transitive dependency). The remaining transitive dependencies were:

- In STUDENT: program_code → program_name, department_code, and department_code → department_name.
- In SECTION: subject_code → subject_name, units, subject_department, and instructor_email → instructor names and department.
- In PAYMENT: method_name → method_available.

Each determinant was moved into its own table. A surrogate key (an auto-increment integer ID) was then added to every table, and the natural keys (student_no, program_code, subject_code, and so on) were kept as UNIQUE columns. The final 3NF tables are:

Table 3.2. Final tables in Third Normal Form

3NF table	Primary key	Came from
departments	department_id	STUDENT, SECTION
programs	program_id	STUDENT
students	student_id	STUDENT
subjects	subject_id	SECTION
instructors	instructor_id	SECTION
sections	section_id	SECTION
enrollments	enrollment_id	ENROLLMENT
payment_methods	payment_method_id	PAYMENT
payments	payment_id	PAYMENT

No department_id is stored in students, because a student's department is already found through programs. This avoids a transitive dependency and keeps the department in one place.

3.5 Data Dictionary

The data dictionary lists every field of the nine tables with its data type, key, nullability and constraints. All primary keys are INT AUTO_INCREMENT. NN means NOT NULL.

Table 3.3. Data dictionary of the departments table

Field	Data type	Key	Null	Constraints / Default	Description
department_id	INT	PK	NN	AUTO_INCREMENT	Unique ID of the department
department_code	VARCHAR(10)		NN	UNIQUE	Short code, e.g., CCE
department_name	VARCHAR(100)		NN	UNIQUE	Full name of the department

Table 3.4. Data dictionary of the programs table

Field	Data type	Key	Null	Constraints / Default	Description
program_id	INT	PK	NN	AUTO_INCREMENT	Unique ID of the program
program_code	VARCHAR(10)		NN	UNIQUE	Short code, e.g., BSIT
program_name	VARCHAR(100)		NN	UNIQUE	Full name of the program
department_id	INT	FK	NN	References departments	Department that offers the program

Table 3.5. Data dictionary of the subjects table

Field	Data type	Key	Null	Constraints / Default	Description
subject_id	INT	PK	NN	AUTO_INCREMENT	Unique ID of the subject
subject_code	VARCHAR(10)		NN	UNIQUE	Subject code, e.g., IT6
subject_name	VARCHAR(100)		NN		Descriptive title of the subject

units	**DECIMAL(2, 1)**		**NN**	**CHECK (units > 0)**	**Number of units, e.g., 3.0**
department_id	**INT**	**FK**	**Null**	**References departments**	**Department that handles the subject**

Table 3.6. Data dictionary of the instructors table

Field	Data type	Key	Null	Constraints / Default	Description
instructor_id	**INT**	**PK**	**NN**	**AUTO_INCREMENT**	**Unique ID of the instructor**
first_name	**VARCHAR(50)**		**NN**		**Instructor's first name**
middle_name	**VARCHAR(50)**		**Null**		**Instructor's middle name**
last_name	**VARCHAR(50)**		**NN**		**Instructor's last name**
email	**VARCHAR(100)**		**Null**	**UNIQUE**	**Instructor's email address**
department_id	**INT**	**FK**	**Null**	**References departments**	**Department the instructor belongs to**

Table 3.7. Data dictionary of the students table

Field	Data type	Key	Null	Constraints / Default	Description
student_id	**INT**	**PK**	**NN**	**AUTO_INCREMENT**	**Unique ID of the student**
student_no	**VARCHAR(10)**		**NN**	**UNIQUE**	**School-issued student number, e.g., 2025-0001**
first_name	**VARCHAR(50)**		**NN**		**Student's first name**

middle_name	VARCHAR(50)		Null		Student's middle name
last_name	VARCHAR(50)		NN		Student's last name
gender	ENUM		Null	'Male', 'Female', 'Other'	Student's gender
email	VARCHAR(100)		Null	UNIQUE	Student's email address
contact_no	VARCHAR(20)		Null		Mobile or phone number
program_id	INT	FK	Null	References programs	Program the student is taking
year_level	TINYINT		NN	CHECK (1–5)	Current year level
status	ENUM		NN	'Active', 'Inactive', 'Graduated', 'Dropped'; default 'Active'	Current student status
date_registered	TIMESTAMP		NN	DEFAULT CURRENT_TIMESTAMP	Date and time the record was created

Table 3.8. Data dictionary of the sections table

Field	Data type	Key	Null	Constraints / Default	Description
section_id	INT	PK	NN	AUTO_INCREMENT	Unique ID of the section
section_code	VARCHAR(10)		NN	UNIQUE with school_year and semester	Section code, e.g., IT6-A
subject_id	INT	FK	NN	References subjects	Subject offered in the section

instructor_id	INT	FK	Null	References instructors	Instructor assigned to the section
school_year	VARCHAR(9)		NN		School year, e.g., 2026-2027
semester	ENUM		NN	'1st', '2nd', 'Summer'	Semester of the offering
schedule_day	VARCHAR(10)		Null		Class days, e.g., MWF
start_time	TIME		Null		Class start time
end_time	TIME		Null	CHECK (end_time > start_time)	Class end time
room	VARCHAR(10)		Null		Room, e.g., R301
max_capacity	INT		NN	CHECK (> 0); default 40	Maximum number of students

Table 3.9. Data dictionary of the enrollments table

Field	Data type	Key	Null	Constraints / Default	Description
enrollment_id	INT	PK	NN	AUTO_INCREMENT	Unique ID of the enrollment
student_id	INT	FK	NN	References students; UNIQUE with section_id	Enrolled student
section_id	INT	FK	NN	References sections	Section the student enrolled in

enrollment_datetime	**TIMESTAMP**		**NN**	**DEFAULT CURRENT_TIMESTAMP**	**Date and time of enrollment**
status	**ENUM**		**NN**	**'Enrolled', 'Dropped', 'Completed'; default 'Enrolled'**	**Status of the enrollment**

Table 3.10. Data dictionary of the payment_methods table

Field	Data type	Key	Null	Constraints / Default	Description
payment_method_id	**INT**	**PK**	**NN**	**AUTO_INCREMENT**	**Unique ID of the payment method**
method_name	**VARCHAR(50)**		**NN**	**UNIQUE**	**Name, e.g., Cash, GCash**
method_available	**BOOLEAN**		**NN**	**DEFAULT TRUE**	**Whether the method is currently accepted**

Table 3.11. Data dictionary of the payments table

Field	Data type	Key	Null	Constraints / Default	Description
payment_id	**INT**	**PK**	**NN**	**AUTO_INCREMENT**	**Unique ID of the payment**
student_id	**INT**	**FK**	**NN**	**References students**	**Student who paid**
school_year	**VARCHAR(9)**		**NN**		**School year the payment is for**

semester	**ENUM**		**NN**	**'1st', '2nd', 'Summer'**	**Semester the payment is for**
amount	**DECIMAL(10,2)**		**NN**	**CHECK (amount > 0)**	**Amount paid in pesos**
payment_method_id	**INT**	**FK**	**NN**	**References payment_methods**	**Method used to pay**
payment_datetime	**TIMESTAMP**		**NN**	**DEFAULT CURRENT_TIMESTAMP**	**Date and time of payment**

3.6 Primary Keys, Foreign Keys, and Constraints

Every table has an INT AUTO_INCREMENT primary key, ten foreign keys enforce the relationships, and UNIQUE, CHECK, DEFAULT and ENUM constraints protect the data entered.

3.6.1 Primary Keys

Table 3.12. Primary keys of each table

Table	Primary key
departments	**department_id**
programs	**program_id**
subjects	**subject_id**
instructors	**instructor_id**
students	**student_id**
sections	**section_id**
enrollments	**enrollment_id**
payment_methods	**payment_method_id**
payments	**payment_id**

3.6.2 Foreign Keys

ON UPDATE CASCADE is used for all foreign keys, so a changed parent ID is updated in the child rows. ON DELETE RESTRICT blocks deleting a parent that still has records (for example, a student with payments). ON DELETE SET NULL is used only where the link is optional.

Table 3.13. Foreign key constraints

Constraint name	Child table (column)	Parent table (column)	On delete
fk_programs_dept	programs (department_id)	departments (department_id)	RESTRICT
fk_subjects_dept	subjects (department_id)	departments (department_id)	SET NULL
fk_instructors_dept	instructors (department_id)	departments (department_id)	SET NULL
fk_students_program	students (program_id)	programs (program_id)	SET NULL
fk_sections_subject	sections (subject_id)	subjects (subject_id)	RESTRICT
fk_sections_instructor	sections (instructor_id)	instructors (instructor_id)	SET NULL
fk_enrollments_student	enrollments (student_id)	students (student_id)	RESTRICT
fk_enrollments_section	enrollments (section_id)	sections (section_id)	RESTRICT
fk_payments_student	payments (student_id)	students (student_id)	RESTRICT
fk_payments_method	payments (payment_method_id)	payment_methods (payment_method_id)	RESTRICT

3.6.3 Other Constraints

Table 3.14. Other constraints

Type	Table	Rule	Purpose
UNIQUE	**departments**	**department_code; department_name**	**No duplicate departments**
UNIQUE	**programs**	**program_code; program_name**	**No duplicate programs**
UNIQUE	**subjects**	**subject_code**	**No duplicate subject codes**
UNIQUE	**instructors**	**email**	**One account per email**
UNIQUE	**students**	**student_no; email**	**No duplicate student numbers or emails**
UNIQUE	**sections**	**(section_code, school_year, semester)**	**A section code is used once per term**
UNIQUE	**enrollments**	**(student_id, section_id)**	**A student cannot enroll twice in the same section**
UNIQUE	**payment_methods**	**method_name**	**No duplicate methods**
CHECK	**subjects**	**units > 0**	**Units must be positive**
CHECK	**students**	**year_level BETWEEN 1 AND 5**	**Valid year level only**
CHECK	**sections**	**max_capacity > 0**	**Capacity must be positive**
CHECK	**sections**	**end_time > start_time**	**Class must end after it starts**
CHECK	**payments**	**amount > 0**	**No zero or negative payments**
ENUM	**students**	**gender, status**	**Only listed values are accepted**
ENUM	**sections, payments**	**semester**	**'1st', '2nd' or 'Summer' only**
ENUM	**enrollments**	**status**	**'Enrolled', 'Dropped' or 'Completed' only**
DEFAULT	**students, enrollments, payments**	**CURRENT_TIMESTAMP**	**Date and time are filled in automatically**

DEFAULT	**payment_methods**	**method_available = TRUE**	**New methods are available by default**

The ENUM values, the UNIQUE pairs, the CHECK rules and the ON DELETE actions above are not shown in the ERD. They should be added to the dbdiagram.io diagram, or changed here, so that the ERD and the database match.

3.7 Initial SQL Database Implementation

The database, named enrollment_db, was created in MariaDB 10.11 using the script below. It builds all nine tables with their keys and constraints and loads 42 initial sample records. The same script is submitted as enrollment_db.sql and can be imported in phpMyAdmin (Import tab) or run with mysql -u root -p < enrollment_db.sql.